



Orviohub Settings Implementation Plan

This implementation should establish the settings foundation before building the full Inventory application:

```
User account
├── Organization / Workspace
│   ├── Organization settings
│   ├── Applications
│   │   └── Application settings
│   └── Branches
│       └── Branch settings
```

The organization is the business container. Applications control product behavior, and branches control location-specific operations. This matches the existing workspace architecture, which separates workspace profile, memberships, applications, branches, billing, and audit logs.^[1]

1. Settings Boundaries

Use three settings levels.

Organization settings

Applies to the entire organization:

- Organization name.
- Display name.
- Logo.
- Business type.
- Business category.
- Email.
- Phone.
- Country.
- State.
- City.
- Full address.

- Currency.
- Timezone.
- Organization slug.
- Business description.
- Default notification settings.
- Enabled applications.
- Team and role management.
- Billing.
- Organization audit log.
- Data export.
- Archive and deletion request.

Application settings

Applies only to an application such as Inventory:

- Product configuration.
- Stock rules.
- Sales rules.
- Customer and supplier settings.
- Receipt settings.
- Application roles.
- Application permissions.
- Application notifications.
- Application onboarding.
- Application audit log.

Branch settings

Applies only to one branch:

- Branch name.
- Branch code.
- Branch logo, if needed.
- Branch address.
- Branch phone and email.
- Branch manager.
- Opening hours.
- Branch-specific receipt information.

- Branch staff.
- Branch permissions.
- Branch operational settings.
- Branch status.
- Branch activity.

Do not put personal profile settings here. Passwords, personal email, 2FA, private sessions, and personal security remain under the central account area.^[2]

2. Recommended Frontend Structure

Use the workspace context in the URL or secure application context.

Shared workspace settings

```
home.orviohub.localhost/workspaces/:workspaceId/settings
```

Recommended pages:

```
/settings
/settings/general
/settings/business
/settings/contact
/settings/branding
/settings/localization
/settings/applications
/settings/members
/settings/roles
/settings/invitations
/settings/billing
/settings/notifications
/settings/audit-log
/settings/data
```

Inventory settings

```
inventory.orviohub.localhost/settings
```

Recommended pages:

```
/settings
/settings/products
/settings/stock
/settings/sales
/settings/receipts
/settings/team
/settings/roles
```

```
/settings/notifications  
/settings/audit-log
```

Branch settings

```
inventory.orviohub.localhost/settings/branches/:branchId
```

Recommended pages:

```
/settings/branches/:branchId  
/settings/branches/:branchId/general  
/settings/branches/:branchId/contact  
/settings/branches/:branchId/hours  
/settings/branches/:branchId/team  
/settings/branches/:branchId/permissions  
/settings/branches/:branchId/operations  
/settings/branches/:branchId/activity
```

For the MVP, branch settings can be embedded inside Inventory settings. However, keep the routes and backend services branch-scoped so that the system does not need to be redesigned later.

3. Convex Data Model

Workspace fields

The existing workspace model should be extended with organization settings.

```
workspaces: defineTable({  
  name: v.string(),  
  displayName: v.optional(v.string()),  
  slug: v.string(),  
  type: v.string(),  
  category: v.optional(v.string()),  
  description: v.optional(v.string()),  
  
  ownerId: v.id("users"),  
  
  logoUrl: v.optional(v.string()),  
  logoStorageId: v.optional(v.id("_storage")),  
  
  email: v.optional(v.string()),  
  phone: v.optional(v.string()),  
  
  country: v.string(),  
  state: v.optional(v.string()),  
  city: v.optional(v.string()),  
  addressLine1: v.optional(v.string()),  
  addressLine2: v.optional(v.string()),  
  postalCode: v.optional(v.string()),
```

```

    currency: v.string(),
    timezone: v.string(),

    status: v.string(),
    createdAt: v.number(),
    updatedAt: v.number(),
  })
  .index("by_owner", ["ownerId"])
  .index("by_slug", ["slug"])
  .index("by_status", ["status"])

```

Do not store all possible settings directly on the workspace forever. Keep general business identity there and place configurable preferences in dedicated settings records.

Workspace settings

```

workspaceSettings: defineTable({
  workspaceId: v.id("workspaces"),

  defaultLanguage: v.optional(v.string()),
  dateFormat: v.optional(v.string()),
  numberFormat: v.optional(v.string()),
  weekStartsOn: v.optional(v.string()),

  taxEnabled: v.boolean(),
  taxDisplayMode: v.optional(v.string()),

  defaultReceiptFooter: v.optional(v.string()),
  defaultNotificationMode: v.optional(v.string()),

  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_workspace", ["workspaceId"])

```

Workspace branding

```

workspaceBranding: defineTable({
  workspaceId: v.id("workspaces"),

  logoUrl: v.optional(v.string()),
  logoStorageId: v.optional(v.id("_storage")),
  faviconUrl: v.optional(v.string()),
  primaryColor: v.optional(v.string()),
  secondaryColor: v.optional(v.string()),

  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_workspace", ["workspaceId"])

```

For the MVP, use the organization logo across applications. Add application-specific branding later if required.

Branches

Branches should include an optional application key because future products may use branches differently.

```
branches: defineTable({
  workspaceId: v.id("workspaces"),
  productKey: v.optional(v.string()),

  name: v.string(),
  code: v.optional(v.string()),
  description: v.optional(v.string()),

  logoUrl: v.optional(v.string()),

  email: v.optional(v.string()),
  phone: v.optional(v.string()),

  addressLine1: v.optional(v.string()),
  addressLine2: v.optional(v.string()),
  country: v.optional(v.string()),
  state: v.optional(v.string()),
  city: v.optional(v.string()),
  postalCode: v.optional(v.string()),

  timezone: v.optional(v.string()),
  currency: v.optional(v.string()),

  isPrimary: v.boolean(),
  status: v.string(),

  createdBy: v.id("users"),
  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_workspace", ["workspaceId"])
.index("by_workspace_status", ["workspaceId", "status"])
.index("by_workspace_product", ["workspaceId", "productKey"])
.index("by_workspace_code", ["workspaceId", "code"])
```

Branch settings

```
branchSettings: defineTable({
  branchId: v.id("branches"),
  workspaceId: v.id("workspaces"),
  productKey: v.optional(v.string()),

  openingHours: v.optional(v.any()),
  receiptFooter: v.optional(v.string()),
  negativeStockAllowed: v.optional(v.boolean()),
```

```

    lowStockThreshold: v.optional(v.number()),

    createdAt: v.number(),
    updatedAt: v.number(),
  })
  .index("by_branch", ["branchId"])
  .index("by_workspace", ["workspaceId"])

```

Do not place Inventory-specific settings in the generic branch record. Keep them in application-specific settings or branch settings identified by `productKey`.

Application settings

```

applicationSettings: defineTable({
  workspaceId: v.id("workspaces"),
  productKey: v.string(),

  displayName: v.optional(v.string()),
  enabled: v.boolean(),

  settings: v.any(),

  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_workspace", ["workspaceId"])
.index("by_workspace_product", ["workspaceId", "productKey"])

```

For stricter validation, create separate tables later:

```

inventorySettings
taskSettings
bookingSettings
gymSettings

```

For the current demo stage, a validated `settings` object is acceptable, but the backend must still validate allowed fields.

4. Permissions Model

Settings access must be permission-based, not based only on a frontend role name.

Organization permissions

```

workspace.view
workspace.update
workspace.manage_branding
workspace.manage_contact
workspace.manage_localization
workspace.manage_members

```

```
workspace.manage_roles
workspace.manage_applications
workspace.manage_billing
workspace.view_audit
workspace.export_data
workspace.archive
workspace.delete
```

Application permissions

```
application.view
application.update_settings
application.manage_members
application.manage_roles
application.manage_notifications
application.view_audit
```

Branch permissions

```
branch.view
branch.update
branch.manage_contact
branch.manage_hours
branch.manage_members
branch.manage_permissions
branch.manage_operations
branch.view_activity
branch.archive
```

Suggested role mapping

Role	Organization settings	Application settings	Branch settings
Owner	Full access	Full access	Full access
Workspace Admin	Most settings	Full application access	Manage assigned branches
Workspace Manager	Limited	Application operational settings	Manage assigned branches
Application Manager	No billing or organization deletion	Full application settings	Manage application branches
Branch Manager	No organization settings	Limited	Full assigned branch settings
Staff	Read limited settings	Operational access	Assigned operational access
Viewer	Read-only	Read-only	Read-only

The backend should calculate effective permission from:

```
workspace membership
+ workspace role
```

```
+ application membership
+ application role
+ branch membership
+ branch role
+ explicit permissions
```

This follows the existing workspace authorization model. [\[1\]](#)

5. Organization Settings Implementation

General settings

Fields:

- Organization name.
- Display name.
- Organization type.
- Category.
- Description.
- Slug.

Rules:

- Name is required.
- Name must be trimmed and normalized.
- Slug must be unique.
- Slug changes should be rate-limited.
- Existing references must continue using the immutable workspace ID.
- Slug changes should create an audit event.
- Do not use the slug as the tenant security boundary.

API:

```
GET    /v1/workspaces/:workspaceId/settings/general
PATCH /v1/workspaces/:workspaceId/settings/general
```

Business information

Fields:

- Business category.
- Business registration number, optional.
- Business email.
- Business phone.

- Business description.
- Number of locations.
- Number of employees.
- Main business type.

Rules:

- Validate Nigerian phone formats.
- Normalize email addresses.
- Do not confuse business contact information with the owner's personal profile.
- Record every change.

API:

```
GET    /v1/workspaces/:workspaceId/settings/business
PATCH /v1/workspaces/:workspaceId/settings/business
```

Address settings

Fields:

- Address line one.
- Address line two.
- Country.
- State.
- City.
- Local government area.
- Postal code.
- Landmark.

For the Inventory MVP, store organization headquarters separately from branch addresses.

API:

```
GET    /v1/workspaces/:workspaceId/settings/address
PATCH /v1/workspaces/:workspaceId/settings/address
```

Branding and logo

Features:

- Upload organization logo.
- Preview logo.
- Crop logo.

- Replace logo.
- Remove logo.
- Generate a thumbnail.
- Use logo on receipts and invitations.
- Use logo in application headers.

Rules:

- Validate MIME type.
- Validate file size.
- Reject executable files.
- Store files in protected object storage.
- Do not expose private storage credentials.
- Use signed URLs or public CDN URLs according to visibility.
- Audit upload, replacement, and deletion.

API:

```
POST    /v1/workspaces/:workspaceId/settings/branding/logo
DELETE  /v1/workspaces/:workspaceId/settings/branding/logo
PATCH  /v1/workspaces/:workspaceId/settings/branding
GET     /v1/workspaces/:workspaceId/settings/branding
```

For the low-cost MVP, one organization logo is enough. Do not build a complete media library yet.

Localization

Fields:

- Currency.
- Timezone.
- Country.
- Date format.
- Number format.
- Default language.
- First day of week.

Rules:

- Organization currency controls business transactions.
- User currency preference only controls personal display.
- Changing currency after transactions exist should be restricted.

- If the currency changes, require confirmation and display the effect.
- Timezone changes should affect future timestamps but not rewrite historical records.

API:

```
GET    /v1/workspaces/:workspaceId/settings/localization
PATCH /v1/workspaces/:workspaceId/settings/localization
```

Notification settings

Organization-level preferences:

- Workspace activity.
- Member changes.
- Billing notifications.
- Application alerts.
- Inventory alerts.
- Low-stock alerts.
- Failed synchronization alerts.
- Product announcements.

Security notifications should not be disabled through organization settings.

API:

```
GET    /v1/workspaces/:workspaceId/settings/notifications
PATCH /v1/workspaces/:workspaceId/settings/notifications
```

Application settings

Display:

- Available applications.
- Active applications.
- Suspended applications.
- Application onboarding state.
- Application usage.
- Application billing or entitlement state.

Actions:

- Activate application.
- Deactivate application.
- Suspend application access.

- Restore application access.
- Open application settings.
- Open application audit log.

API:

```
GET    /v1/workspaces/:workspaceId/applications
POST   /v1/workspaces/:workspaceId/applications/:productKey/activate
POST   /v1/workspaces/:workspaceId/applications/:productKey/deactivate
POST   /v1/workspaces/:workspaceId/applications/:productKey/suspend
POST   /v1/workspaces/:workspaceId/applications/:productKey/restore
```

Application activation must verify membership, workspace status, billing status, plan entitlement, application limits, and onboarding state.

6. Inventory Settings

Even though Inventory is currently a demo dashboard, create the settings structure now.

Product settings

Future fields:

- SKU rules.
- Barcode rules.
- Product naming format.
- Category behavior.
- Product archive behavior.
- Price levels.
- Cost visibility.
- Product import defaults.

Stock settings

- Negative stock allowed.
- Low-stock threshold.
- Stock adjustment approval.
- Stock-count behavior.
- Costing method.
- Opening stock behavior.
- Branch transfer rules.

Sales settings

- Payment methods.
- Allow credit sales.
- Allow discounts.
- Require customer for credit sales.
- Sale cancellation rules.
- Return rules.
- Receipt numbering.

Receipt settings

- Receipt logo.
- Business name.
- Address.
- Phone.
- Footer message.
- Tax display.
- Customer information.
- Payment reference.
- Receipt number format.

Inventory permissions

- View products.
- Create products.
- Adjust stock.
- Receive stock.
- Record sales.
- Cancel sales.
- Process returns.
- View cost.
- View profit.
- Export reports.
- Manage members.
- Manage settings.

Stock-changing operations must eventually create stock movements and audit records. The PRD specifically requires that stock cannot be silently edited and posted records must be corrected through reversals or adjustments. ^[1]

7. Branch Settings Implementation

Branch list

Route:

```
inventory.orviohub.localhost/settings/branches
```

Display:

- Branch name.
- Branch code.
- Address.
- Manager.
- Status.
- Primary branch indicator.
- Member count.
- Product count.
- Stock count.
- Last activity.
- Actions.

Actions:

- Create branch.
- Open branch settings.
- Set as primary.
- Suspend branch.
- Restore branch.
- Archive branch.

Branch profile

Fields:

- Branch name.
- Branch code.
- Description.
- Logo.
- Phone.

- Email.
- Address.
- State.
- City.
- Local government area.
- Opening hours.
- Timezone.
- Currency.

Rules:

- Branch name is required.
- Branch code must be unique within the organization.
- At least one active branch must remain for Inventory.
- The primary branch cannot be archived until another branch becomes primary.
- Branch archive must preserve historical sales and stock records.
- Branch deletion should not be available in the MVP.

API:

```
GET    /v1/workspaces/:workspaceId/branches
POST   /v1/workspaces/:workspaceId/branches
GET    /v1/workspaces/:workspaceId/branches/:branchId
PATCH /v1/workspaces/:workspaceId/branches/:branchId
POST   /v1/workspaces/:workspaceId/branches/:branchId/set-primary
POST   /v1/workspaces/:workspaceId/branches/:branchId/suspend
POST   /v1/workspaces/:workspaceId/branches/:branchId/restore
POST   /v1/workspaces/:workspaceId/branches/:branchId/archive
```

Branch team settings

Display:

- Active members.
- Suspended members.
- Branch role.
- Application role.
- Permissions.
- Last activity.
- Invitation status.

Actions:

- Invite staff.

- Assign member to branch.
- Remove branch access.
- Change branch role.
- Suspend branch access.
- Restore branch access.
- Transfer member to another branch.

Branch access changes should preserve workspace membership and access to other applications or branches unless explicitly revoked.

API:

```
GET    /v1/workspaces/:workspaceId/branches/:branchId/members
POST   /v1/workspaces/:workspaceId/branches/:branchId/members
PATCH /v1/workspaces/:workspaceId/branches/:branchId/members/:membershipId
DELETE /v1/workspaces/:workspaceId/branches/:branchId/members/:membershipId
```

Branch operational settings

For Inventory:

- Opening hours.
- Sales availability.
- Negative stock behavior.
- Low-stock threshold.
- Receipt footer.
- Branch payment methods.
- Branch-specific pricing, later.
- Stock transfer permissions.
- Stock adjustment approval.

Do not place global organization billing or personal profile settings at branch level.

8. Validation Rules

Organization validation

- Name: required, trimmed, length-limited.
- Slug: lowercase, unique, safe characters.
- Email: normalized and validated.
- Phone: normalized.
- Country: supported country code.

- State and city: valid for selected country where possible.
- Currency: valid ISO currency code.
- Timezone: valid IANA timezone.
- Logo: allowed MIME type and size.
- Description: length-limited.

Branch validation

- Branch name: required.
- Branch code: unique within workspace.
- Address: optional but validated.
- Phone: normalized.
- Email: normalized.
- Status transitions must be valid.
- Cannot archive the only active primary branch.
- Cannot assign a user to a branch outside the organization.
- Cannot assign Inventory permissions without Inventory application access.

Settings update behavior

Every update should:

1. Authenticate the user.
2. Resolve the workspace.
3. Verify active membership.
4. Verify the specific permission.
5. Validate the payload.
6. Compare current and new values.
7. Apply the mutation.
8. Write an audit event.
9. Send a notification if important.
10. Return the updated settings.

9. Confirmation and Security

Use different protection levels.

Normal changes

For:

- Editing organization address.
- Editing business description.
- Editing logo.
- Updating branch phone.
- Updating opening hours.

Use:

- Save button.
- Unsaved-change warning.
- Success notification.
- Audit event.

Sensitive changes

For:

- Changing organization name.
- Changing slug.
- Changing currency.
- Changing billing settings.
- Changing roles.
- Changing application access.
- Archiving a branch.
- Suspending an organization.

Require:

- Confirmation modal.
- Clear target and consequence.
- Reason where appropriate.
- Recent password re-authentication.
- TOTP for higher-risk actions.

Destructive changes

For:

- Organization deletion.
- Data export.

- Ownership transfer.
- Entitlement override.
- Removing the only owner.

Require:

- Re-authentication.
- TOTP.
- Confirmation phrase.
- Mandatory reason.
- High-severity audit event.
- Cooling-off period where appropriate.

A fixed PIN should not replace password and 2FA. If you add an action PIN later, hash it, rate-limit it, never log it, and bind successful verification to one specific action and target.

10. Audit Events

Use separate event types for organization, application, and branch settings.

Organization events

```
workspace.settings_updated
workspace.profile_updated
workspace.address_updated
workspace.branding_updated
workspace.logo_uploaded
workspace.logo_removed
workspace.localization_updated
workspace.notification_settings_updated
workspace.currency_changed
workspace.slug_changed
workspace.archived
workspace.restored
workspace.deletion_requested
```

Application events

```
application.settings_updated
application.activated
application.deactivated
application.suspended
application.restored
application.onboarding_updated
application.roles_updated
```

Branch events

```
branch.created  
branch.updated  
branch.address_updated  
branch.branding_updated  
branch.hours_updated  
branch.set_primary  
branch.suspended  
branch.restored  
branch.archived  
branch.member_added  
branch.member_removed  
branch.member_role_changed  
branch.permissions_updated
```

Each event should include:

```
actorUserId  
workspaceId  
productKey  
branchId  
eventType  
changedFields  
beforeValues  
afterValues  
reason  
requestId  
ipAddress  
userAgent  
createdAt
```

Never include passwords, tokens, secrets, payment card data, or raw invitation tokens.

11. Fastify Module Structure

```
src/modules/workspaces/  
  workspace.routes.ts  
  workspace.controller.ts  
  workspace.service.ts  
  workspace.repository.ts  
  workspace.schemas.ts  
  workspace.permissions.ts  
  
src/modules/workspace-settings/  
  workspace-settings.routes.ts  
  workspace-settings.controller.ts  
  workspace-settings.service.ts  
  workspace-settings.schemas.ts  
  
src/modules/branding/  
  branding.routes.ts
```

```
branding.service.ts
upload.service.ts

src/modules/applications/
  application-settings.routes.ts
  application-settings.service.ts

src/modules/branches/
  branch.routes.ts
  branch.controller.ts
  branch.service.ts
  branch.repository.ts
  branch.schemas.ts
  branch.permissions.ts

src/modules/audit/
  audit.service.ts
  audit.repository.ts

src/plugins/
  auth.ts
  request-context.ts
  rate-limit.ts
  convex.ts

src/hooks/
  authenticate.ts
  resolve-workspace.ts
  require-membership.ts
  require-application-access.ts
  require-branch-access.ts
  require-permission.ts
  require-recent-auth.ts
```

Request lifecycle

```
Request
→ Assign request ID
→ Authenticate session
→ Resolve workspace
→ Verify membership
→ Verify application access
→ Verify branch access if required
→ Verify permission
→ Validate input
→ Execute mutation
→ Write audit event
→ Send notification
→ Return response
```

This is consistent with the existing Fastify workspace plan. ^[1]

12. Frontend Components

Build reusable components:

```
SettingsLayout
SettingsSidebar
WorkspaceContextHeader
ApplicationContextHeader
BranchSelector
SettingsSection
SettingsForm
LogoUploader
AddressForm
TimezoneSelect
CurrencySelect
OpeningHoursEditor
PermissionMatrix
MemberAccessTable
AuditLogTable
DangerZone
ConfirmationModal
ReauthenticationModal
```

Important UI behavior

- Always display the active organization.
- Display the active application.
- Display the active branch where applicable.
- Warn the user when changing context.
- Do not allow a branch-only setting to appear as organization-wide.
- Show permission-based actions only when authorized.
- Still enforce every permission on the backend.
- Preserve unsaved form data after validation errors.
- Show loading and error states.
- Show audit confirmation after successful saves.

13. Demo Inventory Integration

Since Inventory is currently a demo dashboard, integrate settings gradually.

First demo integration

Implement:

- Organization name in dashboard header.
- Organization logo in navigation.
- Organization currency in sample prices.
- Organization address on sample receipts.
- Branch selector.
- Branch name in dashboard header.
- Branch-specific demo product and sales data.
- Branch settings page.
- Demo team list.
- Branch-specific audit activity.

Important demo rule

Do not pretend that demo data is production inventory data.

Use clearly separated demo records:

```
workspaceId  
branchId  
demoMode: true
```

When real Inventory data is implemented, use the same workspace and branch context but different production tables.

Example dashboard context

```
{  
  workspace: {  
    id: "ws_123",  
    name: "Code X Stores",  
    logoUrl: "...",  
    currency: "NGN"  
  },  
  application: {  
    key: "inventory",  
    status: "active"  
  },  
  branch: {  
    id: "branch_123",  
    name: "Main Store",  
    status: "active"  
  },  
  permissions: [  
    "inventory.view",
```

```
    "branch.view",  
    "branch.manage_members"  
  ]  
}
```

14. Implementation Phases

Phase 1: Foundation

Implement:

- Workspace settings schema.
- Branch schema.
- Application settings schema.
- Indexes.
- Workspace context resolution.
- Permission middleware.
- Audit event service.
- Basic settings navigation.

Deliverable:

User can securely load organization, application, and branch settings based on the active workspace.

Phase 2: Organization settings

Implement:

- General settings.
- Business profile.
- Contact details.
- Address.
- Branding and logo.
- Localization.
- Notification preferences.
- Audit events.

Deliverable:

Owner can update organization settings and see changes across the application.

Phase 3: Branch settings

Implement:

- Branch list.
- Create branch.
- Edit branch.
- Branch address.
- Branch contact.
- Opening hours.
- Set primary branch.
- Suspend and restore.
- Archive branch.
- Branch audit log.

Deliverable:

Organization can manage multiple branches without mixing branch data with organization data.

Phase 4: Application settings

Implement:

- Application list.
- Application activation status.
- Inventory settings.
- Application permissions.
- Application notifications.
- Application onboarding status.
- Application audit log.

Deliverable:

Inventory settings remain isolated from future application settings.

Phase 5: Team settings

Implement:

- Organization members.
- Application membership.
- Branch membership.
- Role assignment.

- Branch transfer.
- Suspension.
- Removal.
- Invitation management.

Deliverable:

A member can belong to the organization but receive access only to selected applications and branches.

Phase 6: Security and dangerous actions

Implement:

- Confirmation modals.
- Recent re-authentication.
- TOTP verification.
- Confirmation phrases.
- Archive workflow.
- Ownership protection.
- Data export workflow.
- Deletion request workflow.
- High-severity audit events.

Deliverable:

Sensitive changes cannot happen through a simple frontend click.

Phase 7: Demo Inventory integration

Implement:

- Organization logo and name in dashboard.
- Branch selector.
- Branch-specific demo state.
- Currency display.
- Address and receipt display.
- Demo audit records.
- Context-aware settings links.

Deliverable:

The demo application proves the organization → application → branch structure before full inventory functionality is built.

15. Testing Plan

Organization tests

- Owner can view settings.
- Admin can edit permitted settings.
- Member cannot edit organization settings.
- Address updates are persisted.
- Logo upload works.
- Invalid logo is rejected.
- Slug collision is rejected.
- Currency update requires confirmation.
- Settings update creates audit event.
- Cross-organization settings access fails.

Application tests

- Inactive application settings are inaccessible.
- Active Inventory settings load correctly.
- Application permission is required.
- Application settings do not affect another application.
- Application suspension blocks application access.
- Application restoration restores access.

Branch tests

- User can view assigned branch.
- User cannot view an unauthorized branch.
- Branch code is unique within workspace.
- Primary branch cannot be archived without replacement.
- Branch suspension blocks branch operations.
- Branch suspension preserves workspace membership.
- Branch archive preserves historical records.
- Branch update creates audit event.

Team tests

- Organization member can lack application access.
- Application member can lack branch access.
- Branch transfer updates access correctly.
- Removed branch member cannot access branch.
- Removed member's historical records remain.
- Owner cannot be removed without ownership transfer.

Security tests

- Forged workspace ID fails.
- Forged branch ID fails.
- Frontend-only permission changes fail.
- Expired re-authentication fails.
- Wrong TOTP fails.
- Confirmation token cannot be reused.
- Confirmation token cannot be used for another target.
- Failed attempts are rate-limited.
- Secrets never appear in logs.
- Audit logs cannot be edited.

The existing workspace implementation plan specifically calls for testing forged workspace IDs, cross-tenant access, unauthorized branch access, session reuse after removal, cache leakage, and audit tampering. ^[1]

16. Final Recommended MVP Scope

Build these first:

1. Organization general settings.
2. Organization logo and address.
3. Organization localization.
4. Branch CRUD and branch settings.
5. Branch selector.
6. Organization, application, and branch permissions.
7. Organization and branch audit logs.
8. Inventory demo integration.
9. Confirmation and re-authentication for sensitive changes.
10. Team access structure.

Defer these:

- Advanced media library.
- Application-specific branding.
- Complex tax configuration.
- Multi-currency transaction support.
- Branch-level custom domains.
- Full organization deletion.
- Bulk settings import.
- Two-person administrative approval.
- Advanced analytics.
- Custom workflow automation.

The key architectural decision is to create branch settings now, even before the full Inventory application is implemented. That gives the demo dashboard the correct context and prevents future data from being incorrectly stored at the organization level.

*
**

1. docs/Workspace Management Implementation Plan.pdf

2. docs/User Personal Profile Management Plan.pdf